Lab0 实验报告

一、问题回答

1. Git 基本用法

（1）你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

暂时没有，但是之后可以尝试了。

（2）思考一下，Git 为什么要设计“暂存-提交”两个步骤？

有了单独的暂存功能，就可以随时撤销或修改暂存的文件，减少只因为微小修改而重复提交的次数。

（3）git branch 和 git branch -a 的区别是什么？查阅资料并回答。

```
$ git branch
  feature
* main
```

```
$ git branch -a
  feature
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

这是我在终端输入两种命令行的结果。git branch 只展示本地分支，而 git branch -a 既能显示本地分支又能显示远程追踪分支。（对本次实验好像没有什么影响，不知道之后 的实验会不会用到）。

2. 相关网页阅读

（1）Commit message 和 Change log 编写指南

该文主要介绍了 commit message 的作用、基本格式和标准写法，规范使用 commit message 会提高提交记录的可读性，有利于维护项目。

（2）Gitflow 使用规范

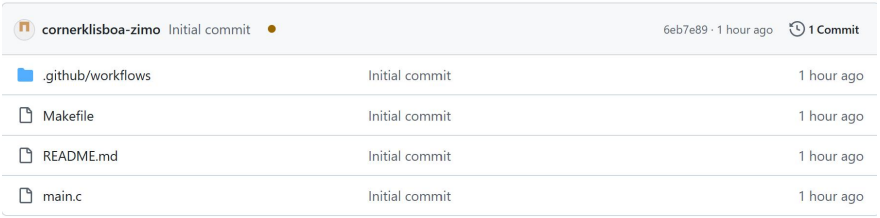
该文主要介绍了 Gitflow 各个关键分支的工作流程，说明了各个分支之间的关系，并给出了使用示例。

（3）为什么要学习 Git？

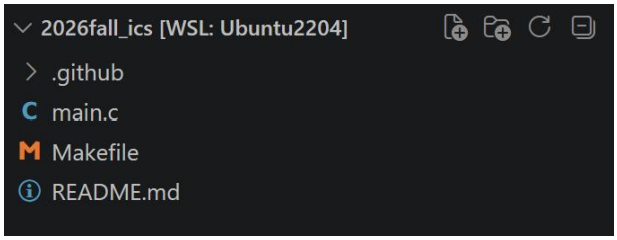
小项目可能体现不出来，但是较大的或者需要团队合作的项目就能体现出 Git 的重要性。使用 Git 可以方便回溯代码版本，便于维护代码。借助 Gitflow 的分支机制，团队多人可以互不干扰、各自开发。最后，用 Git 建立一个丰富规范的 Github 主页有利于 cs 学生求职（哎）。

二、实验步骤

1. 成功建立个人仓库



2. 成功克隆仓库到本地（忘记截图相关的命令行了）



3. 修改 main.c 并提交一次



4. 建立 feature 分支，修改 main.c 文件

```
$ git branch feature
$ git checkout feature
Switched to branch 'feature'
$ git branch
* feature
  main
```

5. 将 feature 分支 merge 到 main 分支，出现冲突

```
$ git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

```
int main()
{
    // @TODO: print a sentence you want.
    printf("Hello, world!\n");
    printf("This is my lab0 for ics2026:)\n");
采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
<<<<<<< HEAD (当前更改)
    printf("edited by main\n");
=======
    printf("edited by feature\n");
>>>>>>> feature (传入的更改)
}
```

6. 选择保留双方更改，同时留下两行代码，再次提交，冲突解决

